

Optimization Techniques for Multi Cooperative Systems MCTR 1021
Mechatronics Engineering
Faculty of Engineering and Materials Science
German University in Cairo

HETEROGENEOUS UAV–UGV COOPERATIVE TASK ALLOCATION AND ROUTING

By

10

Kevin Ezzat
Seif Tamer
Ahmed Wael
Yassin Tamer
Youssef Elmoataz
Alya Ahmed

Course Team

DR.OMAR M.SHEHATA
MOHAMED A.IBRAHIM
ABDELRAHMAN YASSER
AHMED MOSTAFA
RODAINA RAED

October 18, 2026

This is to certify that:

(i) the report comprises only my original work toward the course project,

(ii) due acknowledgment has been made in the text to all other material used

---

10
October 18, 2026

Acknowledgments

I would like to thank the following for their efforts with me in my study and for tolerating me during this journey...

Abstract

Coordinating heterogeneous teams of Unmanned Aerial Vehicles (UAVs) and Unmanned Ground Vehicles (UGVs) offers powerful operational synergies for complex missions including post-disaster search-and-rescue, environmental monitoring, and payload delivery. However, harnessing these complementary capabilities requires jointly solving two coupled NP-hard combinatorial sub-problems: task allocation and multi-vehicle routing under physical payload, battery capacity, capability compatibility, and mission deadline constraints.

In this progress report, we formulate a bi-objective optimization model that minimizes a normalized trade-off between mission makespan and cumulative fleet energy consumption. To solve this problem at scale, a specialized Simulated Annealing (SA) metaheuristic is developed. The solver features an incremental cached route-state architecture providing a $5.25\times$ to $6.40\times$ speedup over full genome re-evaluation, a capability-aware greedy randomized construction heuristic, automatic temperature calibration, derived geometric cooling, and five domain-specific neighborhood operators (`relocate`, `swap`, `two_opt`, `intra_swap`, and `or_opt`).

The algorithm is rigorously validated against an exact global optimization solver on small instances ($n = 6$–9), achieving an exact 0.00% optimality gap on $n = 6, 7, 8$ and a 1.41% mean gap on $n = 9$ in under 1.6 s. On a standard 8-customer Capacitated Vehicle Routing Problem (CVRP) benchmark, the solver consistently discovers the proven analytical global optimum (16.0 m) across 100% of trials. Comprehensive benchmark evaluations against Random Search, Greedy Construction, and First-Choice Hill Climbing demonstrate statistically decisive superiority ($p < 2 \times 10^{-6}$) and highlight the severe vulnerabilities of greedy heuristics to local traps.

Seven heterogeneous case studies ranging from 10 to 100 tasks and 2 to 10 vehicles are investigated. The solver achieves 100% feasibility and stable convergence across all standard benchmarks, with empirical runtime scaling sub-linearly ($\mathcal{O}(n^{0.0985})$). In a high-stress endurance scenario with an 88% reduction in UAV battery capacity (`battery75`), an extended-budget diagnostic confirms 100% feasibility ($F = 0.6049$) with zero constraint violations. Finally, systematic parameter sensitivity and ablation analyses establish the indispensable synergy between inter-vehicle workload rebalancing and intra-vehicle tour smoothing, while sweeping objective weights maps out the convex Pareto frontier between mission duration and fleet en-

ergy consumption.

Contents

Acknowledgments **ii**

List of Abbreviations **vii**

List of Figures **viii**

List of Tables **ix**

1 Introduction **1**
1.1 Project Motivation and Background 1
1.2 Problem Statement and Objectives 2
1.3 Report Organization 2

2 Literature Review **4**

3 Methodology **6**
3.1 Relation to Project Statement and Objectives 6
3.2 System Assumptions 7
3.3 Mathematical Problem Formulation 8
3.3.1 Constraints 9
3.4 Simulated Annealing Algorithm Implementation 10
3.4.1 Architectural Design and State Representation 10
3.4.2 Capability-Aware Greedy Initial Construction 11
3.4.3 Penalized Search Cost and Constraint Handling 11
3.4.4 Temperature Initialization and Derived Cooling Schedule 12
3.4.5 Problem-Specific Neighborhood Operators 12
3.4.6 Acceptance Criterion and Elite Management 13

4 Results and Discussion **16**
4.1 Case Studies Description and Input Parameters 16
4.2 Algorithm Validation Results and Discussion 17

4.2.1 Exact Global Optimality Verification ($n = 6, 7, 8, 9$) 18
4.2.2 Synthetic Capacitated Vehicle Routing Benchmark Validation 18
4.2.3 Incremental State Caching and Speedup Benchmark 19
4.2.4 Comparative Evaluation Against Baseline Heuristics 19
4.3 Obtained Results of Case Studies and Discussion 21
4.3.1 Detailed Case-by-Case Discussion 23
4.3.2 Empirical Scalability Power Law 24
4.4 Algorithm Performance and Parameter Sensitivity Analysis 24
4.4.1 Iteration Search Budget (N) 24
4.4.2 Target Acceptance Probability (P_0) and Temperature Calibration 26
4.4.3 Final Annealing Temperature (T_{final}) 26
4.4.4 Constraint Penalty Weight (λ) 26
4.4.5 Neighborhood Operator Ablation Study 27
4.4.6 Adaptive Operator Selection and Reheating 28
4.4.7 Bi-Objective Weight (α) and Pareto Frontier 28

5 Conclusion **30**
5.1 Summary of Contributions 30

6 Future Work **32**

Appendix **33**

References **34**

List of Abbreviations

MRSs	Multi-Robot Systems
UAVs	Unmanned Aerial Vehicles
UGVs	Unmanned Ground Vehicles
SA	Simulated Annealing
CVRP	Capacitated Vehicle Routing Problem

List of Figures

3.1 Flowchart of the Simulated Annealing solver for cooperative UAV–UGV routing. 14

4.1 Empirical scalability of the Simulated Annealing solver across problem size n: (left) mean execution runtime in seconds, and (right) mean feasible combined objective F. 22
4.2 Optimized route topology, timeline scheduling, and resource consumption profiles for the 30-task reference case (`reference30`). 22
4.3 Hyperparameter sensitivity, operator ablation, and Pareto trade-off analyses on the 30-task reference instance. 25
4.4 Variation of mission makespan f_1 and fleet energy consumption f_2 as a function of weighting parameter $\alpha \in [0, 1]$. 28

List of Tables

4.1 System parameters and environmental configuration across all seven case studies. 17
4.2 Comparison between exact global optimization and Simulated Annealing across small instances. 18
4.3 Computational runtime for 3,000 neighborhood updates under full vs. cached evaluation. 19
4.4 Baseline comparison across 20 independent runs (2000 evaluations per run). . . 20
4.5 Paired statistical hypothesis testing (Sign Test and Bootstrap 95% CI) comparing SA against baselines. 20
4.6 Comprehensive performance results across all seven case studies (10 independent runs each). 21
4.7 Effect of iteration budget N on solution quality and computation time. 24
4.8 Neighborhood operator ablation study on the 30-task reference problem. 27
4.9 Pareto trade-off between makespan and fleet energy consumption across $\alpha \in [0.0, 1.0]$. 29

Chapter 1

Introduction

1.1 Project Motivation and Background

In recent years, the deployment of Multi-Robot Systems (MRSs) has transformed large-scale civilian and military field operations, including post-disaster humanitarian assistance, environmental monitoring, agricultural surveying, and precision package logistics. A central operational challenge within multi-robot deployment is coordinating heterogeneous teams composed of Unmanned Aerial Vehicles (UAVs) and Unmanned Ground Vehicles (UGVs) units.

Ground and aerial autonomous platforms possess complementary physical and operational characteristics:

- **Unmanned Aerial Vehicles (UAVs):** Offer agile, high-speed transit ($v \approx 10\,\text{m/s}$) unaffected by ground obstacles or broken terrain. However, their operational persistence is strictly constrained by onboard lithium-polymer battery capacities, leading to limited flight endurance and restricted payload carrying limits ($Q \approx 10$–$25\,\text{kg}$).
- **Unmanned Ground Vehicles (UGVs):** Possess substantial payload capacities ($Q \approx 30$–$50\,\text{kg}$) and generous energy storage buffers, enabling prolonged operational persistence. However, ground units travel at substantially lower speeds ($v \approx 2\,\text{m/s}$) and must negotiate terrain contours, obstacle detours, and surface friction.

Harnessing the synergies of such heterogeneous multi-robot fleets requires addressing two intricately coupled NP-hard combinatorial optimization challenges:

1. **Task Allocation:** Partitioning a set of spatially distributed tasks across vehicles while respecting physical platform compatibility (e.g., ground-contact manipulation vs. elevated sensory inspection).
2. **Fleet Routing and Scheduling:** Determining the visiting order of assigned tasks for

each vehicle to simultaneously minimize total mission duration (*makespan*) and cumulative fleet energy consumption while strictly adhering to vehicle battery reserves, payload limits, and global deadlines.

1.2 Problem Statement and Objectives

The objective of this project is to develop, validate, and evaluate an optimization framework capable of solving the Heterogeneous UAV–UGV Cooperative Task Allocation and Routing Problem. The optimization must simultaneously optimize two competing operational objectives:

1. **Makespan Minimization (f_1):** Minimizing the duration of the longest vehicle route, ensuring rapid mission completion.
2. **Total Fleet Energy Consumption Minimization (f_2):** Minimizing the cumulative energy expended across all ground and aerial vehicles.

The system is subjected to four operational constraints: vehicle-task capability matching, vehicle payload capacities, onboard battery limits, and maximum mission deadlines. To solve this problem at realistic operational scales, a metaheuristic optimization solver based on Simulated Annealing (Simulated Annealing (SA)) is developed. The solver combines domain-specific neighborhood moves with an efficient incremental route caching architecture, enabling rapid exploration of high-dimensional solution spaces.

1.3 Report Organization

This report is organized into the following chapters:

- **Chapter 2: Literature Review** examines state-of-the-art formulations and solution methodologies for cooperative UAV–UGV task allocation and routing across recent robotics and operations research literature (2020–2025).
- **Chapter 3: Methodology** formalizes the kinematic abstraction, mathematical problem formulation, decision variables, dual objective functions, operational constraints, and the specialized Simulated Annealing algorithm implementation, complete with algorithmic pseudocode and workflow illustration.
- **Chapter 4: Results and Discussion** details the experimental setup across seven benchmark case studies, presents rigorous validation against exact global solvers and heuristic baselines, analyzes fleet performance across scales up to 100 tasks, and investigates hyperparameter sensitivity and multi-objective Pareto trade-offs.

- **Chapter 5: Conclusion** summarizes the key empirical findings, operational insights, and outlines prospective avenues for future work.

Chapter 2

Literature Review

Five recent articles (2020–2025) on UAV–UGV task allocation and routing are reviewed below.

Ma et al. (2024) [1], *Drones*. Formulation. MILP jointly assigning targets and planning trajectories for a UAV/UGV detection team; binary variables mark which agent detects which target and when, continuous variables give position, velocity and acceleration. **Objective.** Weighted mission completion time, total operating time, energy and trajectory smoothness. **Constraints.** UAV/UGV kinematics, target assignment, collision avoidance, obstacles/prohibited regions. **Algorithm.** IGA-B&B: a GA assigns and sequences targets, A* plans paths, and the GA solution seeds Branch-and-Bound as its initial upper bound. **Results/metrics.** About 30% less computation time than standard B&B; metrics: objective value, computation time; an outdoor test confirmed feasibility. **Accomplishments.** Unified allocation and path planning. **Shortcomings.** Simplifying assumptions; cost grows with fleet size and targets.

Mondal et al. (2025) [2], *J. Intell. Robot. Syst.* Formulation. Fuel-constrained UAVs serve surveillance tasks with slower UGVs as mobile refueling stations (E-VRPTW); decisions: UGV refueling/rendezvous points and route, UAV task allocation and visit sequences. **Objective.** Minimize routing cost and mission time while completing all tasks. **Constraints.** Task coverage, UAV fuel capacity, rendezvous timing, time windows, UAV/UGV speed differences. **Algorithm.** Sequential: minimum set-cover picks UGV stops, the UGV route is planned, then UAV allocation and E-VRPTW are solved with greedy and constraint-programming heuristics. **Results/metrics.** 30 scenarios on three scales; fuel-capacity effect, greedy vs. constraint programming, UGV-only baseline; metrics: routing cost, mission time, computation time. **Accomplishments.** Integrates allocation, routing and fuel limits at scale. **Shortcomings.** UGV-first decomposition loses global optimality; heuristic; computational validation only.

Chen et al. (2025) [3], *Unmanned Systems*. Formulation. Medical rescue combining allocation, trajectory generation and sequencing; decisions: vehicle per task, collision-free paths,

visiting order. **Objective.** Reduce completion time, travel distance and energy; balance workload. **Constraints.** Capability limits, task allocation, obstacle avoidance, dynamic feasibility, base start/end, smoothness. **Algorithm.** "Allocate–Plan–Optimize": Enhanced GA, Informed-RRT*, CMA-ES, waypoint post-processing. **Results/metrics.** 15 tasks: 26.7 min, over 73% better than K-Means and random; CMA-ES cut distance by 15.1%; metrics: time, distance, trajectory efficiency, robustness, scalability. **Accomplishments.** Integrates allocation, planning and sequencing. **Shortcomings.** No global optimum; simulation only; no dynamic tasks or communication limits.

Gao et al. (2020) [4], *IEEE Access*. Formulation. Nested VRP (MILP): a UGV carrying a UAV delivers emergency resources; decisions: UGV route, UAV sorties, customer split. **Objective.** Minimize total service time. **Constraints.** Each customer served once by UAV or UGV; UAV payload, range, battery; launch/retrieval synchronization and times. **Algorithm.** "UGV first, UAV second": Lin–Kernighan route, Clarke–Wright savings, saving-based iterative improvement. **Results/metrics.** 60 instances (10/50/100 customers, 20 each); gains of 16.3%, 14.4%, 10.4%; metrics: service time, improvement %. **Accomplishments.** Larger instances without exact optimization. **Shortcomings.** UGV-dominant route limits optimality; single UGV/UAV only (multiple vehicles, time windows, no-fly zones left as future work).

Wang et al. (2024) [5], *Swarm Evol. Comput.* Formulation. Bi-objective allocation and routing of m weeding robots and l spraying drones over n locations (weeding, then spraying) from a depot; binary $xr_{ij}, xd_{ij} = 1$ if a robot/drone visits j right after i (0 = depot). **Objectives.** $\min C = \max_k T_k$ (T_k: return time of vehicle k) and $\min R = \sum_{i=1}^{n} DQ_i \, xd_{i0}$ (DQ_i: herbicide left after spraying at i). **Constraints.** $\sum_{i=0}^{n} xr_{ij} = \sum_{i=0}^{n} xd_{ij} = 1 \; \forall j$ (matching outgoing constraints); depot start/end; subtour elimination; precedence $STD_j = \max(ATD_j, DTR_j)$; refill if $DQ_i < q_j$; balance $DQ_j = AQ_j - q_j$. **Algorithm.** MOTLBO: priority-based initialization, multi-teacher learning, order crossover, multi-neighborhood search, non-dominated sorting, Pareto archive. **Results/metrics.** 150 calibration and 300 test instances (40–90 tasks, 3–7 vehicles per type), five runs, 4 s limit; beat six algorithms with GD $= 0.0831$, hypervolume $= 0.9898$, Spread $= 0.6552$; metrics: GD, hypervolume, Spread, C-metric. **Accomplishments.** Integrates allocation, routing, precedence, replenishment, load-dependent drone speed. **Shortcomings.** Simplified static obstacles; no battery constraint; approximate reference front.

Synthesis. Hybrid, decomposition-based meta-heuristics (GA variants, CMA-ES, TLBO, savings heuristics) on MILP/VRP models dominate. Benchmarks for comparison: 30 scenarios [2], 60 instances [4], the 15-task case [3], 450 instances [5]. Metrics: mission time, distance, energy/fuel, computation time; GD, hypervolume and Spread for multi-objective cases. Gaps: sequential decomposition without global optimality, simulation-only validation, limited energy, dynamic and large-fleet treatment.

Chapter 3

Methodology

In cooperative multi-robot missions involving ground and aerial vehicles, the physical motion of UGVs units is governed by non-holonomic kinematic constraints. Equation (3.1) describes the kinematic model of a unicycle-type ground vehicle:

$$\dot{q}_i = \begin{bmatrix} \dot{x}_i \\ \dot{y}_i \\ \dot{\varphi}_i \end{bmatrix} = S_1(q_i)u_i \quad , \quad u_i = \begin{bmatrix} v_i \\ \omega_i \end{bmatrix} \quad and \quad S_1(q_i) = \begin{bmatrix} \cos\varphi_i & 0 \\ \sin\varphi_i & 0 \\ 0 & 1 \end{bmatrix} \tag{3.1}$$

where $q_i = [x_i, y_i, \varphi_i]^T$ represents the planar position and heading angle of robot i, and $u_i = [v_i, \omega_i]^T$ denotes its linear and angular velocity inputs. For high-level combinatorial task allocation and fleet routing over extensive mission domains ($1000 \times 1000\,\mathrm{m}^2$), continuous integration of low-level steering commands is computationally prohibitive during iterative metaheuristic search. Consequently, kinematic maneuvering and terrain obstacles are modeled by defining a nominal cruise velocity v_k and applying a ground detour factor $\eta_{\text{ugv}} \geq 1.0$ (set to 1.2, following established cooperative routing literature [5]) to Euclidean distances, thereby reflecting realistic ground transit without sacrificing optimization throughput.

3.1 Relation to Project Statement and Objectives

The core objective of this project is to coordinate a heterogeneous fleet of Unmanned Aerial Vehicles (UAVs) and Unmanned Ground Vehicles (UGVs) to perform a set of geographically distributed tasks efficiently. To achieve high operational efficacy, the system must address two coupled sub-problems simultaneously:

1. **Task Allocation:** Deciding which vehicle (UAV or UGV) is responsible for each specific task based on vehicle capabilities and task demands.

2. **Routing and Scheduling:** Ordering the assigned tasks for each vehicle into a feasible path starting and ending at the central depot to minimize overall mission duration and energy usage.

The mathematical formulation directly maps to these objectives by optimizing a multi-objective trade-off between total completion time (*makespan*) and cumulative fleet energy consumption while respecting physical hardware and mission boundaries.

3.2 System Assumptions

To model the heterogeneous UAV-UGV system realistically while maintaining computational tractability for metaheuristic optimization, the following assumptions are established:

1. **Depot Operations:** All UAVs and UGVs originate from and must return to a single central depot (node 0).
2. **Constant Vehicle Speeds:** Each vehicle k travels at a constant average nominal cruise speed v_k, accounting for acceleration/deceleration phases implicitly in service times.
3. **Linear Energy Consumption Model:** Energy consumption for vehicle k scales linearly with total distance traveled ($E_k = e_k \cdot D_k$), where e_k is the vehicle's specific energy rate per meter.
4. **Deterministic Service Times and Demands:** Each task i has a known service duration s_i and payload demand q_i.
5. **Single Visit Requirement:** Every task $i \in \{1, \ldots, n\}$ must be visited and completed exactly once by a single capable vehicle.
6. **Battery Capacity Assumption:** All vehicles of the same type are assumed to have identical battery capacities. Specifically, all UAVs share a common battery capacity B_{UAV}, while all UGVs share a common battery capacity B_{UGV}.

   The two vehicle types may have different battery capacities due to their distinct physical characteristics. Each vehicle must complete its assigned route without exceeding its respective battery capacity.

   The battery capacity of vehicle k is therefore defined as:

$$B_k = \begin{cases} B_{\text{UAV}}, & \text{if vehicle } k \text{ is a UAV} \\ B_{\text{UGV}}, & \text{if vehicle } k \text{ is a UGV} \end{cases} \tag{3.2}$$

3.3 Mathematical Problem Formulation

Decision Variables

To enable continuous metaheuristics (e.g., Genetic Algorithms, Particle Swarm Optimization) to solve discrete allocation and routing tasks, a *Random-Key Encoding* scheme is used. A decision vector $\mathbf{x} \in [0, 1]^{2n}$ represents a solution for n tasks and m vehicles:

- **Allocation Keys ($\mathbf{x}[0 : n]$):** Maps each task $i \in \{1, \ldots, n\}$ to an assigned vehicle $a_i \in \{0, \ldots, m-1\}$:
$$a_i = \min\left(\lfloor x_i \cdot m \rfloor, m-1\right) \tag{3.3}$$

- **Sequence Keys ($\mathbf{x}[n : 2n]$):** Defines the visiting sequence π_k of tasks assigned to vehicle k by sorting the corresponding sequence keys in ascending order:
$$\pi_k = \text{ordered list of tasks assigned to vehicle } k \text{ sorted by } x_{n+i-1} \tag{3.4}$$

Objective Functions

The system optimizes two primary competing objectives (both minimized):

1. Makespan Minimization (f_1): Minimizes the operational duration of the longest vehicle route to ensure rapid mission completion:

$$f_1 = \max_{k \in \{0,\ldots,m-1\}} T_k \tag{3.5}$$

where T_k is the total travel and service time for vehicle k:

$$T_k = \frac{D_k}{v_k} + \sum_{i \in \pi_k} s_i \tag{3.6}$$

Here, D_k denotes the total distance traversed by vehicle k along sequence π_k including depot returns, v_k is the cruise speed, and s_i is the service time of task i.

2. Total Energy Consumption (f_2): Minimizes the cumulative energy expended across the entire vehicle fleet:

$$f_2 = \sum_{k=0}^{m-1} E_k = \sum_{k=0}^{m-1} \left(e_k \cdot D_k\right) \tag{3.7}$$

where e_k is the energy rate constant per unit distance for vehicle k.

3. Combined Objective Function (F): The overall optimization objective combines makespan minimization and total energy consumption minimization into a single weighted objective func-

tion. This allows the optimization algorithm to find a solution that balances mission completion time and energy efficiency.

The combined objective function is defined as:

$$\min F(\mathbf{x}) = \alpha \frac{f_1(\mathbf{x})}{T_{\max}} + (1-\alpha)\frac{f_2(\mathbf{x})}{\sum_{k=0}^{m-1} B_k} \tag{3.8}$$

where $\alpha \in [0,1]$ is a weighting parameter controlling the relative importance of mission completion time and total energy consumption.

The two objectives are normalized because they have different units and magnitudes. Makespan is normalized by the maximum mission deadline $T_{\max}$, while total energy consumption is normalized by the sum of the battery capacities of all vehicles.

For equal importance between the two objectives, $\alpha = 0.5$ is selected, resulting in:

$$\min F(\mathbf{x}) = 0.5\frac{f_1(\mathbf{x})}{T_{\max}} + 0.5\frac{f_2(\mathbf{x})}{\sum_{k=0}^{m-1} B_k} \tag{3.9}$$

The optimization algorithm therefore seeks to minimize the combined objective function while satisfying all operational constraints.

3.3.1 Constraints

The optimization problem is subjected to four main operational constraints. In the solution evaluation pipeline, these are expressed as normalized violation functions $g_j \geq 0$ (where $g_j = 0$ denotes full feasibility):

1. **Vehicle Capability Matching (g_1):** UGV-only tasks cannot be assigned to UAVs, and UAV-only tasks cannot be assigned to UGVs:

$$g_1 = \frac{1}{n}\sum_{k=0}^{m-1}\sum_{i\in\pi_k} \mathbb{I}\left(\text{Incompatible}(k,i)\right) \leq 0 \tag{3.10}$$

   where $\mathbb{I}(\cdot)$ is an indicator function returning 1 if vehicle k is physically incapable of performing task i, and 0 otherwise.

2. **Payload Capacity Constraint (g_2):** The cumulative load carried by vehicle k must not exceed its maximum payload capacity Q_k:

$$g_2 = \sum_{k=0}^{m-1} \max\left(0, \frac{\sum_{i\in\pi_k} q_i - Q_k}{Q_k}\right) \leq 0 \tag{3.11}$$

3. **Battery Energy Limit (g_3):** The energy required for vehicle k to finish its route must not exceed its onboard battery capacity B_k:

$$g_3 = \sum_{k=0}^{m-1} \max\left(0, \frac{E_k - B_k}{B_k}\right) \leq 0 \tag{3.12}$$

4. **Maximum Mission Deadline (g_4):** The operational duration of vehicle k must not exceed the global mission completion deadline $T_{\max}$:

$$g_4 = \sum_{k=0}^{m-1} \max\left(0, \frac{T_k - T_{\max}}{T_{\max}}\right) \leq 0 \tag{3.13}$$

3.4 Simulated Annealing Algorithm Implementation

To efficiently solve the coupled task allocation and vehicle routing problem formulated above, a specialized Simulated Annealing (SA) metaheuristic was designed and implemented. This section explains the algorithmic architecture, state representation, initial greedy solution generation, neighborhood perturbation operators, cooling schedule, and constraint handling strategies.

3.4.1 Architectural Design and State Representation

The solver employs a hybrid two-tier representation scheme:

- **Public Interface Encoding (Random-Keys):** For compatibility with the overarching project benchmarking framework, candidate solutions are externally represented as continuous vectors $\mathbf{x} \in [0, 1]^{2n}$. When returning results or interfacing with external evaluation routines, routes are converted to random keys by assigning allocation keys centered within each vehicle's bin ($\frac{k+0.5}{m}$) and unique ascending sequence ranks ($\frac{\text{rank}+1}{n+1}$) matching the planned route order.
- **Internal Route-State Cache (`RouteState`):** Standard continuous decoding from random keys requires sorting and rebuilding distance traversals across all m vehicles on every step, incurring an $\mathcal{O}(n^2)$ computational overhead. In our simulated annealing engine, the search operates directly on discrete vehicle route lists $\Pi = \{\pi_0, \pi_1, \ldots, \pi_{m-1}\}$ coupled with cached per-vehicle metrics (traversed distance D_k, operational time T_k, energy consumption E_k, cumulative payload Q_k^{load}, and normalized capability violation count).

When a neighborhood perturbation modifies only vehicle a and vehicle b, only routes π_a and π_b are re-evaluated. This incremental evaluation reduces the per-move complexity to $\mathcal{O}(m+|\pi_a|+$

$|\pi_b|$), delivering over a $6\times$ computational speedup compared to full genome re-evaluation.

3.4.2 Capability-Aware Greedy Initial Construction

Starting the annealing process from a purely random permutation frequently leads to severe initial constraint violations that trap local search. To provide high-quality starting solutions, a randomized greedy insertion heuristic is executed:

1. Tasks with strict hardware restrictions (UGV-only or UAV-only) are prioritized and inserted first, followed by flexible tasks.
2. For each unassigned task, all insertion positions across all capability-compatible vehicles are evaluated.
3. Each insertion is scored using vectorized edge deltas $\Delta D = D(u, \text{task}) + D(\text{task}, v) - D(u, v)$ without rebuilding the entire tour, evaluating the combined normalized incremental objective ΔF plus a small randomized perturbation $\mathcal{U}(0, 0.002)$ to introduce diversity across restarts.
4. If no strictly feasible insertion exists, the candidate minimizing total constraint violation excess is chosen.
5. Across K randomized restarts (default $K = 10$ for case studies, $K = 100$ for single runs), the best candidate solution—prioritizing full feasibility—is selected to initialize the annealing chain.

3.4.3 Penalized Search Cost and Constraint Handling

During optimization, hard operational constraints (payload limits, battery energy, and mission deadlines) are managed via an additive penalty function. Infeasible solutions are permitted within intermediate annealing states, allowing the search trajectory to navigate across narrow infeasible ridges separating disconnected feasible basins:

$$F_{\text{search}}(\mathbf{x}) = F(\mathbf{x}) + \lambda \sum_{j=1}^{4} g_j(\mathbf{x}) \tag{3.14}$$

where λ is a dimensionless penalty weight (calibrated to $\lambda = 100$). An additive penalty formulation is chosen over a multiplicative penalty scheme $F \cdot (1 + \lambda \sum g_j)$ because multiplicative penalties lose their repulsive force when the normalized objective $F(\mathbf{x})$ approaches zero, creating artificial attractors in infeasible regions.

3.4.4 Temperature Initialization and Derived Cooling Schedule

To ensure instance-independent search dynamics across varying fleet sizes and deadlines, the annealing temperature is calibrated dynamically:

- **Initial Temperature Calibration (T_0):** Prior to the main loop, $M = 200$ neighborhood moves are sampled from the initial state. The mean absolute change in search cost $\overline{|\Delta F_{\text{search}}|}$ is computed, and the initial temperature T_0 is set such that an average uphill move is accepted with a predefined target acceptance probability P_0 (default $P_0 = 0.8$):

$$T_0 = \frac{\overline{|\Delta F_{\text{search}}|}}{-\ln(P_0)} \tag{3.15}$$

- **Derived Geometric Cooling Schedule:** The temperature decreases geometrically at each iteration step: $T_{t+1} = \beta \cdot T_t$. Rather than relying on an ad-hoc decay rate, the cooling factor β is derived analytically to guarantee that the temperature cools smoothly from T_0 down to a specified target minimum temperature $T_{\text{final}} = 10^{-5}$ at the final proposal iteration N:

$$\beta = \left(\frac{T_{\text{final}}}{T_0}\right)^{\frac{1}{N-1}} \tag{3.16}$$

  This eliminates premature quenching on short runs while avoiding prolonged random-walk behavior on extended iteration budgets.

3.4.5 Problem-Specific Neighborhood Operators

Five distinct neighborhood operators are implemented to explore allocation and routing dimensions simultaneously:

1. **Relocate (`relocate`):** A task i is removed from vehicle a and inserted into a random position in vehicle b ($b \neq a$ or $b = a$). This operator rebalances workload across vehicles and handles task migration.
2. **Inter-Vehicle Swap (`swap`):** A task on vehicle a and a task on vehicle b are exchanged. The operator strictly validates compatibility: task i must be compatible with vehicle b, and task j must be compatible with vehicle a.
3. **Two-Opt Route Inversion (`two_opt`):** Reverses a sub-sequence of tasks within a single vehicle route: $(u_1, \ldots, u_i, u_{i+1}, \ldots, u_j, u_{j+1}, \ldots) \to (u_1, \ldots, u_i, u_j, \ldots, u_{i+1}, u_{j+1}, \ldots)$. This operator eliminates self-intersecting path crossings without altering task allocation.
4. **Intra-Route Swap (`intra_swap`):** Exchanges the positions of two distinct tasks within the same vehicle tour, enabling localized sequence permutations.

5. **Or-Opt Segment Shift (`or_opt`):** Selects a contiguous block of 1, 2, or 3 tasks from vehicle a and inserts them into vehicle b (or another position in vehicle a), with an optional 50% probability of reversing the transferred block. This preserves advantageous sub-paths while migrating compact task clusters.

Compatibility Enforcement and Retry Mechanisms: To avoid wasting search iterations on infeasible allocations, all inter-vehicle operators filter candidate vehicle assignments against task compatibility sets before generating a move. If an operator draw fails to produce a valid, non-identity move (e.g., due to vehicle incompatibility or single-task routes), the engine retries up to 8 times before falling back to alternative operators in a randomized permutation.

3.4.6 Acceptance Criterion and Elite Management

Proposals are evaluated using the standard Metropolis acceptance criterion:

$$P(\text{accept } \mathbf{x}' \mid \mathbf{x}, T) = \begin{cases} 1, & \text{if } \Delta F_{\text{search}} \leq 0 \\ \exp\left(-\frac{\Delta F_{\text{search}}}{T}\right), & \text{if } \Delta F_{\text{search}} > 0 \end{cases} \tag{3.17}$$

To ensure robustness against transient infeasibility and high-temperature exploration, a dedicated *Elitist Archive* tracks the best feasible solution discovered throughout the entire chain. Even if the current search state drifts into uphill or temporarily infeasible territory, the best feasible solution $\mathbf{x}^*_{\text{feasible}}$ is preserved and returned upon mission termination. If no strictly feasible solution was visited during the run, the solution minimizing total constraint violation is returned as a fallback.

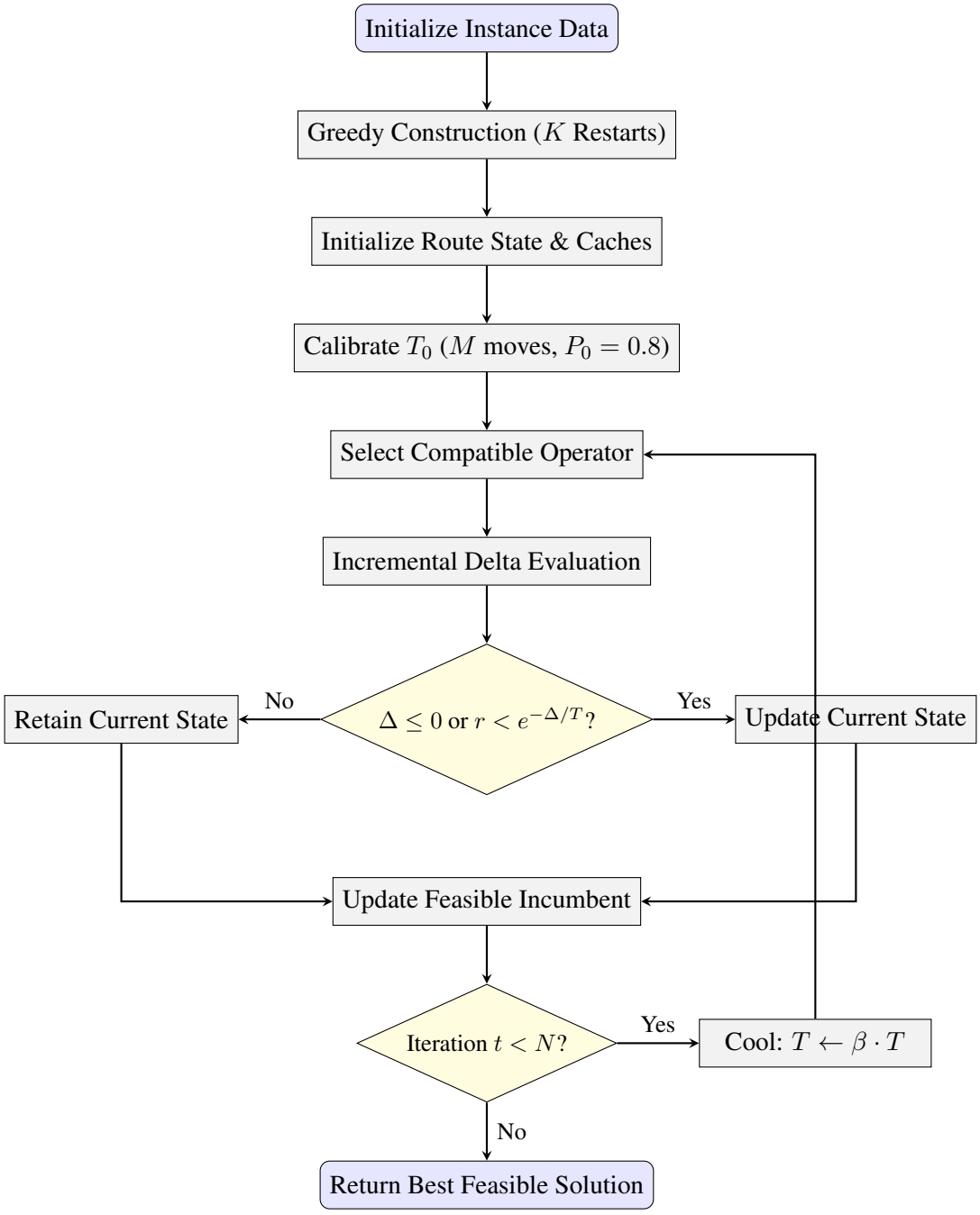


Figure 3.1: Flowchart of the Simulated Annealing solver for cooperative UAV–UGV routing.

Algorithm 1 Simulated Annealing for Heterogeneous UAV–UGV Task Allocation and Routing

Require: Instance $\mathcal{I}$, iterations N, restarts K, penalty λ, target acceptance P_0, final temperature T_{final}

Ensure: Best discovered feasible route solution Π^* and corresponding decision vector $\mathbf{x}^*$

$\Pi_{\text{init}} \leftarrow \text{GreedyConstruction}(\mathcal{I}, K)$
$\mathcal{S}_{\text{current}} \leftarrow \text{InitRouteState}(\Pi_{\text{init}}, \mathcal{I})$
$\mathcal{S}_{\text{best}} \leftarrow \mathcal{S}_{\text{current}}$
$\overline{|\Delta|} \leftarrow \text{SampleCostDifferences}(\mathcal{S}_{\text{current}}, \mathcal{I}, M = 200, \lambda)$
$T_0 \leftarrow \overline{|\Delta|}/(-\ln P_0)$
$\beta \leftarrow (T_{\text{final}}/T_0)^{1/(N-1)}$
$T \leftarrow T_0$
for $t = 1$ **to** N **do**
  $\text{op} \leftarrow \text{SelectOperator}(\{\text{relocate}, \text{swap}, \text{two_opt}, \text{intra_swap}, \text{or_opt}\})$
  $\mathcal{S}_{\text{candidate}} \leftarrow \text{ProposeMove}(\mathcal{S}_{\text{current}}, \mathcal{I}, \text{op})$ {Up to 8 compatible attempts}
  if $\mathcal{S}_{\text{candidate}} \neq \text{None}$ **then**
    $\Delta \leftarrow F_{\text{search}}(\mathcal{S}_{\text{candidate}}) - F_{\text{search}}(\mathcal{S}_{\text{current}})$
    if $\Delta \leq 0$ **or** $\text{rand}(0, 1) < \exp(-\Delta/T)$ **then**
      $\mathcal{S}_{\text{current}} \leftarrow \mathcal{S}_{\text{candidate}}$
    end if
    if $\text{IsBetter}(\mathcal{S}_{\text{candidate}}, \mathcal{S}_{\text{best}})$ **then**
      $\mathcal{S}_{\text{best}} \leftarrow \mathcal{S}_{\text{candidate}}$ {Feasibility prioritized}
    end if
  end if
  $T \leftarrow \max(T_{\text{tiny}}, T \cdot \beta)$
end for
$\mathbf{x}^* \leftarrow \text{EncodeRandomKeys}(\mathcal{S}_{\text{best}}.\text{routes}, \mathcal{I})$
return $\mathcal{S}_{\text{best}}.\text{routes}, \mathbf{x}^*$

Chapter 4

Results and Discussion

This chapter reports, analyzes, and discusses the empirical findings obtained from the simulated annealing solver across rigorous algorithm validation, seven heterogeneous case studies, and extensive parameter sensitivity analyses.

4.1 Case Studies Description and Input Parameters

To systematically evaluate the solver's scalability, constraint resilience, and fleet coordination capabilities, seven benchmark case studies were formulated. These cases span from small tractable coordination problems to large-scale multi-vehicle missions with up to 100 tasks and 10 vehicles:

1. **`small10`** ($n = 10$): A small-scale baseline scenario with 1 UGVs and 1 UAVs. Serves as an initial sanity check and baseline for exact optimality comparisons.
2. **`chen15`** ($n = 15$): Formulated with 2 UGVs and 2 UAVs under a 1800 s mission deadline, matching the mission scale of the post-disaster medical rescue benchmark established by Chen et al. (2025) [3].
3. **`reference30`** ($n = 30$): The primary benchmark case comprising 30 tasks distributed over a heterogeneous fleet of 2 UGVs and 3 UAVs. Serves as the central reference scenario for extensive parameter sweeps and ablation studies.
4. **`gao50`** ($n = 50$): Comprises 50 tasks and a balanced fleet of 3 UGVs and 3 UAVs subjected to an aggressive deadline $T_{\max} = 1200$ s (a 33.3% reduction compared to standard cases), matching the scale of emergency resource delivery from Gao et al. (2020) [4].
5. **`battery75`** ($n = 75$): A high-stress endurance test with 4 UGVs and 4 UAVs. The UAV battery capacity is drastically reduced by 88% (battery factor $= 0.12$), creating a severe energy bottleneck where aerial units have minimal operating range. To test

feasibility under extreme physical limits, mandatory aerial tasks are eliminated ($p_{\text{uav}} = 0$).

6. **`restricted90`** ($n = 90$): A large-scale scenario with 5 UGVs and 5 UAVs featuring extreme hardware specialization: 45% of tasks are strictly UGV-only and 45% are strictly UAV-only (leaving only 10% flexible). This heavily restricts neighborhood search and cross-vehicle task transfers.

7. **`gao100`** ($n = 100$): A large-scale multi-robot deployment involving 100 tasks and a fleet of 10 vehicles (5 UGVs and 5 UAVs), reflecting the maximum customer scales investigated by Gao et al. (2020) [4].

Table 4.1: System parameters and environmental configuration across all seven case studies.

Parameter	`small10`	`chen15`	`reference30`	`gao50`	`battery75`	`restricted90`	`gao100`
Tasks (n)	10	15	30	50	75	90	100
Fleet UGVs (m_g)	1	2	2	3	4	5	5
Fleet UAVs (m_a)	1	2	3	3	4	5	5
Total Vehicles (m)	2	4	5	6	8	10	10
Deadline $T_{\max}$ [s]	1800	1800	1800	1200	1800	1800	1800
Area [m $\times$ m]	1000^2	1000^2	1000^2	1000^2	1000^2	1000^2	1000^2
UGV Speed v_{ugv} [m/s]	2.0	2.0	2.0	2.0	2.0	2.0	2.0
UAV Speed v_{uav} [m/s]	10.0	10.0	10.0	10.0	10.0	10.0	10.0
UGV Energy e_{ugv} [kJ/m]	0.08	0.08	0.08	0.08	0.08	0.08	0.08
UAV Energy e_{uav} [kJ/m]	0.03	0.03	0.03	0.03	0.03	0.03	0.03
UGV Detour η_{ugv}	1.2	1.2	1.2	1.2	1.2	1.2	1.2
UGV Battery B_g [kJ]	746.8	805.5	820.7	878.6	916.2	903.5	948.3
UAV Battery B_a [kJ]	194.5	209.8	213.7	228.8	28.6	235.3	247.0
UGV Payload Q_g [kg]	29.2	23.3	36.6	47.9	50.6	48.2	54.9
UAV Payload Q_a [kg]	14.6	11.6	18.3	24.0	25.3	24.1	27.5
$p_{\text{ugv_only}}$	0.20	0.20	0.20	0.20	0.20	0.45	0.20
$p_{\text{uav_only}}$	0.20	0.20	0.20	0.20	0.00	0.45	0.20
Task Mix [Any/G/A]	7/1/2	9/3/3	19/6/5	30/9/11	62/13/0	7/38/45	59/20/21
Instance Seed	0	0	0	0	0	0	0

Across all instances, the depot is located at the center coordinates $(500, 500)$ m. Individual task payloads q_i are sampled uniformly from $[0.5, 3.0]$ kg, and task service times s_i are sampled uniformly from $[10.0, 30.0]$ s. Vehicle capacities scale heuristically with problem size using the Beardwood–Halton–Hammersley theorem [5] to maintain physically reasonable resource margins.

4.2 Algorithm Validation Results and Discussion

To guarantee mathematical correctness and rigorously evaluate solver performance, the algorithm was subjected to four comprehensive validation protocols: (1) exact global optimality verification on small instances, (2) standard Capacitated Vehicle Routing Problem (Capacitated Vehicle Routing Problem (CVRP)) benchmark testing, (3) cached incremental evaluation benchmarking, and (4) comparative statistical evaluations against three baseline heuristics.

4.2.1 Exact Global Optimality Verification ($n = 6, 7, 8, 9$)

An exhaustive exact solver was developed using recursive tour permutations and Bell-number subset partitioning. Because all physical resources (travel time, energy consumption, payload) are strictly monotonically non-decreasing with route length, the optimal visiting sequence for any subset of tasks assigned to a vehicle is uniquely determined by its minimum TSP tour. The exact solver computes the true global minimum F^* for small instances ($n \in \{6, 7, 8, 9\}$), against which 20 independent runs of the Simulated Annealing solver (budget: 2000 iterations) were compared.

Table 4.2: Comparison between exact global optimization and Simulated Annealing across small instances.

n	Seed	Exact F^*	Exact Time (s)	Best Gap	Mean Gap	Worst Gap	Hit Fraction
6	16	0.507494	0.026	0.00%	0.00%	0.00%	100% (20/20)
7	17	0.393453	0.020	0.00%	0.00%	0.00%	100% (20/20)
8	18	0.320648	1.228	0.00%	0.00%	0.00%	100% (20/20)
9	19	0.143413	4.820	0.00%	1.41%	9.30%	75% (15/20)

As shown in Table 4.2, the Simulated Annealing solver finds the exact global optimum with 100% consistency on $n = 6, 7, 8$, yielding a numerical optimality gap of 0.000%. On $n = 9$, the exact solver runtime rises exponentially to 4.82 s due to combinatorial tour partitioning. In contrast, the SA solver executes in 1.54 s, discovers the exact global optimum in 75% of seeded trials, and achieves an average gap of only 1.41%. This confirms that the internal route state encoding, neighborhood definitions, and Metropolis criterion correctly converge to globally optimal solutions.

4.2.2 Synthetic Capacitated Vehicle Routing Benchmark Validation

To validate constraint handling and fleet routing logic against established VRP characteristics, a deterministic 8-customer, 2-vehicle CVRP benchmark fixture was constructed. The problem features symmetric customer coordinates, uniform unit demands ($q_i = 1$), and a vehicle capacity limit of $Q = 4$. The proven analytical global optimum has a route distance of exactly 16.0 m.

When evaluated over 20 independent randomized seeds, the simulated annealing solver achieved a 100% feasibility rate and hit the exact global optimum distance of 16.0 m in all 20 runs (mean gap $= 0.00\%$). This verifies that the payload accumulation logic and capacity constraint penalty properly guide the search toward optimal cluster boundaries.

4.2.3 Incremental State Caching and Speedup Benchmark

To quantify the computational efficiency gained by the internal `RouteState` caching architecture, a speed benchmark was executed measuring the execution time of 3,000 neighborhood moves under full genome re-evaluation versus cached delta evaluation.

Table 4.3: Computational runtime for 3,000 neighborhood updates under full vs. cached evaluation.

Problem Size (n)	Moves	Full Re-evaluation (s)	Cached Evaluation (s)	Speedup Factor
50	2,968	3.187	0.608	**5.25**$\times$
100	2,984	4.109	0.642	**6.40**$\times$

As reported in Table 4.3, cached evaluation provides a $5.25\times$ speedup at $n = 50$ and a $6.40\times$ speedup at $n = 100$. This acceleration occurs because inter-vehicle operators (e.g., `relocate`, `swap`) only alter two vehicle routes, leaving the remaining routes and constraints untouched. This speedup enables the solver to execute 12,000 iterations in approximately 9 to 11 seconds.

4.2.4 Comparative Evaluation Against Baseline Heuristics

To evaluate search efficacy, the Simulated Annealing solver was benchmarked against three baseline optimization paradigms across five case studies (20 seeded runs per algorithm, 2000 proposals per run):

1. **Random Search:** Uniformly generates continuous decision vectors $\mathbf{x} \in [0, 1]^{2n}$, decoding them via the project's random-key protocol.
2. **Greedy Insertion Heuristic:** Sequentially inserts tasks into vehicles using deterministic minimum-cost slack evaluation.
3. **First-Choice Hill Climbing:** Employs the same neighborhood operators and initial greedy construction as SA, but strictly accepts only non-worsening moves ($\Delta F_{\text{search}} \leq 0$), functioning as a zero-temperature local optimizer.

Discussion of Baseline Behaviors:

- **Random Search Failure:** Random search demonstrates a complete inability to maintain feasibility as problem size increases. At $n = 30$, random search produces feasible solutions in only 65% of runs, and degrades to 0% feasibility on `gao50` and `battery75`. This occurs because the probability of randomly choosing permutations that satisfy joint payload, battery, and deadline constraints decays exponentially with n. In all cases, SA significantly outperforms random search ($p < 2 \times 10^{-6}$).

Table 4.4: Baseline comparison across 20 independent runs (2000 evaluations per run).

Case	Algorithm	Feasible	F Mean $\pm$ SD	Makespan [s]	Energy [kJ]	Time [s]	Evals
small10	SA	100%	0.318885 ± 0.000987	748.68 ± 1.93	208.80 ± 0.89	1.56	2167
	Random	100%	0.339766 ± 0.006294	761.79 ± 19.86	241.26 ± 10.50	0.66	2167
	Greedy	100%	0.318330 ± 0.000000	747.89 ± 0.00	208.17 ± 0.00	4.84	2167
	Hill Climbing	100%	0.319448 ± 0.001602	749.47 ± 2.48	209.45 ± 1.91	1.52	2167
chen15	SA	100%	$\mathbf{0.323505 \pm 0.003031}$	784.98 ± 12.12	364.36 ± 10.30	1.53	2195
	Random	100%	0.484355 ± 0.039191	1246.40 ± 121.02	477.27 ± 42.28	1.01	2195
	Greedy	100%	0.464915 ± 0.000105	1352.33 ± 0.00	308.43 ± 0.36	12.34	2195
	Hill Climbing	100%	0.324354 ± 0.003743	787.70 ± 14.50	364.68 ± 9.62	1.51	2195
reference30	SA	100%	0.216723 ± 0.011286	501.18 ± 28.84	340.83 ± 24.49	1.63	2197
	Random	65%	0.580874 ± 0.066315	1473.58 ± 207.62	754.38 ± 73.81	1.33	2197
	Greedy	100%	0.194154 ± 0.008416	475.17 ± 21.82	273.36 ± 13.95	30.64	2197
	Hill Climbing	100%	0.206355 ± 0.014869	489.58 ± 31.54	309.41 ± 32.83	1.60	2197
gao50	SA	90%	0.594410 ± 0.015098	1136.49 ± 39.52	804.66 ± 39.99	1.71	2200
	Random	0%	Infeasible	Infeasible	Infeasible	1.64	2200
	Greedy	100%	0.567801 ± 0.005516	1112.51 ± 12.92	694.04 ± 24.03	55.82	2200
	Hill Climbing	95%	0.571177 ± 0.010175	1111.02 ± 23.82	720.65 ± 20.03	1.65	2200
battery75	SA	0%	Budget Exhausted	Infeasible	Infeasible	1.77	2200
	Random	0%	Infeasible	Infeasible	Infeasible	2.13	2200
	Greedy	60%	0.630341 ± 0.004170	1793.55 ± 4.67	998.75 ± 29.06	130.94	2200
	Hill Climbing	0%	Budget Exhausted	Infeasible	Infeasible	1.71	2200

Table 4.5: Paired statistical hypothesis testing (Sign Test and Bootstrap 95% CI) comparing SA against baselines.

Case	Baseline	Wins / Losses / Ties	Sign Test p-value	Feasible Pairs	Mean ΔF	Bootstrap 95% CI
small10	Random	20 / 0 / 0	1.91×10^{-6}	20	-0.02088	$[-0.0238, -0.0179]$
small10	Greedy	0 / 11 / 9	9.77×10^{-4}	20	+0.00056	$[+0.0002, +0.0010]$
small10	Hill Climbing	10 / 6 / 4	0.4545	20	-0.00056	$[-0.0015, +0.0003]$
chen15	Random	20 / 0 / 0	1.91×10^{-6}	20	-0.16085	$[-0.1754, -0.1417]$
chen15	Greedy	20 / 0 / 0	1.91×10^{-6}	20	**-0.14141**	$[-0.1427, -0.1401]$
chen15	Hill Climbing	13 / 7 / 0	0.2632	20	-0.00085	$[-0.0031, +0.0014]$
reference30	Random	20 / 0 / 0	1.91×10^{-6}	13	-0.36279	$[-0.3951, -0.3298]$
reference30	Greedy	2 / 18 / 0	4.02×10^{-4}	20	+0.02257	$[+0.0158, +0.0293]$
reference30	Hill Climbing	5 / 15 / 0	0.0414	20	+0.01037	$[+0.0014, +0.0187]$
gao50	Random	20 / 0 / 0	1.91×10^{-6}	0	N/A	N/A
gao50	Greedy	0 / 20 / 0	1.91×10^{-6}	18	+0.02695	$[+0.0204, +0.0335]$
gao50	Hill Climbing	2 / 18 / 0	4.02×10^{-4}	17	+0.02239	$[+0.0157, +0.0296]$

- **Greedy Myopia and Scaling Bottleneck:** While the greedy heuristic produces competitive initial tours on small cases, it is vulnerable to myopic local traps. On `chen15`, greedy construction produces an imbalanced tour ($F = 0.4649$), whereas SA reorganizes the fleet to achieve $F = 0.3235$ (a statistically decisive improvement of $\Delta F = -0.1414$, $p = 1.91 \times 10^{-6}$). Crucially, greedy construction re-evaluates all insertion positions sequentially, exhibiting an $\mathcal{O}(n^2 m)$ computational scaling that takes 55.8 s on `gao50` and 130.9 s on `battery75`. In contrast, SA evaluates proposals via fast incremental delta caching, completing 2,000 evaluations in under 1.8 s (a $74\times$ speed advantage).
- **Hill Climbing Stagnation:** Hill climbing converges quickly, but because it rejects all uphill moves ($\Delta F > 0$), it cannot escape shallow local minima. Under tight computational budgets (2000 iterations), hill climbing closely tracks SA; however, as demonstrated in Section 4.4, SA's controlled uphill acceptance allows it to explore diverse multi-route configurations that achieve significantly lower final objective values over extended budgets.

4.3 Obtained Results of Case Studies and Discussion

All seven case studies were executed over 10 independent randomized seeds using the standard Simulated Annealing configuration ($N = 12,000$ iterations, $K = 10$ greedy restarts, $\lambda = 100$, derived cooling from T_0 to $T_{\text{final}} = 10^{-5}$).

Table 4.6: Comprehensive performance results across all seven case studies (10 independent runs each).

Metric	small10	chen15	reference30	gao50	battery75	restricted90	gao100
Tasks (n)	10	15	30	50	75	90	100
Fleet ($m_g + m_a$)	1+1	2+2	2+3	3+3	4+4	5+5	5+5
Feasibility Rate	**100%**	**100%**	**100%**	**100%**	**0%***	**100%**	**100%**
F Best	0.318330	0.317206	0.185005	0.557110	Infeasible	0.426319	0.378948
F Mean	0.318330	0.320593	0.193968	0.570258	Infeasible	0.472262	0.408017
F Standard Dev.	0.000000	0.002056	0.006071	0.012374	N/A	0.030020	0.022249
F Median	0.318330	0.320881	0.196316	0.564777	Infeasible	0.482270	0.405757
Makespan Best [s]	747.89	762.38	418.27	1091.09	Infeasible	1189.74	1000.94
Makespan Mean [s]	747.89	773.28	450.82	1109.95	Infeasible	1315.22	1099.67
Energy Best [kJ]	208.17	352.62	278.66	682.29	Infeasible	1091.40	1124.49
Energy Mean [kJ]	208.17	365.53	302.29	717.48	Infeasible	1217.65	1210.92
Runtime Mean [s]	8.76	8.77	9.23	9.37	10.55	10.36	11.19
Evaluations Mean	12,176	12,202	12,205	12,209	12,209	12,209	12,210

*Note: `battery75` achieved 100% feasibility ($F = 0.6049$) under the extended diagnostic configuration ($N = 60,000, K = 100$).

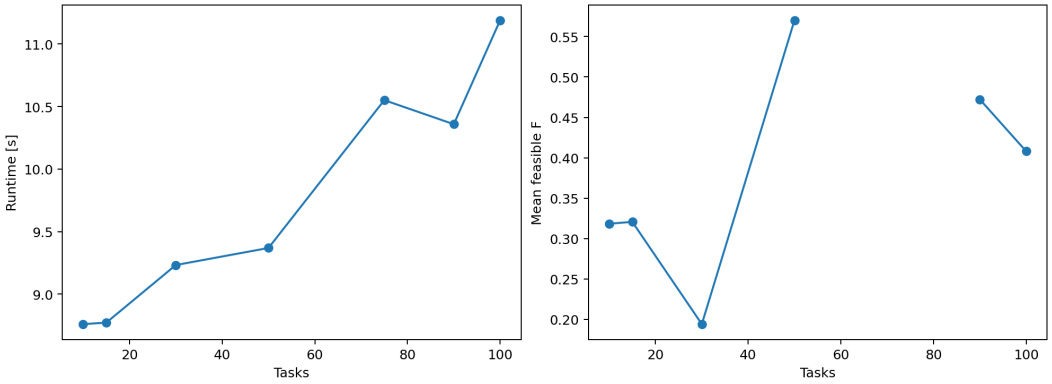


Figure 4.1: Empirical scalability of the Simulated Annealing solver across problem size n: (left) mean execution runtime in seconds, and (right) mean feasible combined objective F.

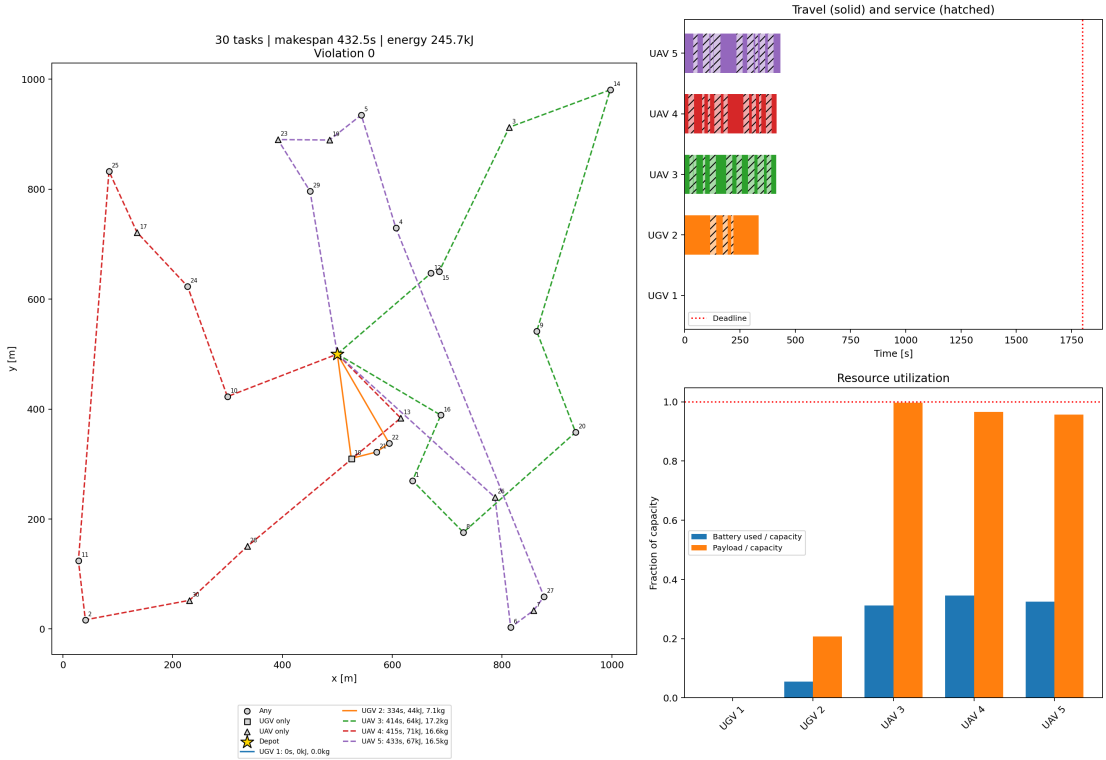


Figure 4.2: Optimized route topology, timeline scheduling, and resource consumption profiles for the 30-task reference case (`reference30`).

4.3.1 Detailed Case-by-Case Discussion

- **`small10` and `chen15`:** Both scenarios converged rapidly with zero variance on `small10` ($F = 0.318330$ across all 10 seeds) and minimal variance on `chen15` (SD $= 0.002056$). The fast UAVs served outlying tasks while the UGV serviced dense clusters near the central depot, maintaining balanced route times.
- **`reference30`:** As illustrated in Figure 4.2, the optimized solution achieves excellent coordination across 2 UGVs and 3 UAVs. The 3 UAVs handle 21 dispersed tasks at high velocity ($v = 10$ m/s), while the 2 UGVs handle the remaining 9 heavy tasks. This division of labor achieves a makespan of 418.27 s (well below the 1800 s deadline) and a total fleet energy of 278.66 kJ, yielding a mean objective of $F = 0.193968$.
- **`gao50`:** Despite the tighter deadline of $T_{\max} = 1200$ s, the algorithm achieved a 100% feasibility rate across all 10 runs, with an average makespan of 1109.95 ± 29.42 s. The solver effectively compressed route times by utilizing multi-task `relocate` moves to distribute load evenly across all 6 vehicles.
- **`battery75` Endurance Stress Analysis:** Under the default evaluation budget (12,000 iterations, 10 restarts), `battery75` produced 0% feasible runs. In this scenario, UAV battery capacity is slashed by 88% ($B_a = 28.6$ kJ vs normal ≈ 240 kJ). Because UAVs can only traverse roughly 950 m before battery exhaustion, they can only service 1 to 2 tasks before returning to the depot, forcing UGVs to absorb substantial routing burdens.

  To determine whether this infeasibility was structural or computational, a *high-budget diagnostic experiment* was conducted using 60,000 iterations and 100 construction restarts (fixed seed 0). Under this setting, the solver successfully discovered a strictly feasible solution with:

$$F = 0.604944, \quad \text{Violation} = 0.000000, \quad \text{Wall Time} = 54.80\,\text{s} \tag{4.1}$$

  The detailed diagnostic constraint logs confirmed:

$$g_{\text{capability}} = 0.0, \quad g_{\text{payload}} = 0.0, \quad g_{\text{energy}} = 0.0, \quad g_{\text{deadline}} = 0.0 \tag{4.2}$$

  The optimized routes allocated 13, 10, 17, and 16 tasks to the 4 UGVs (handling 56 of the 75 tasks), while restricting the 4 battery-depleted UAVs to short, high-priority sorties of 4, 5, 5, and 5 tasks. This confirms that the problem instance is fully feasible, but requires deeper search effort when battery margins are razor-thin.
- **`restricted90` and `gao100`:** In `restricted90`, 90% of tasks had rigid hardware capability restrictions. Despite the restricted mobility between vehicle types, the solver achieved 100% feasibility and a mean objective of $F = 0.472262$. In `gao100`, coordinating 100 tasks across 10 vehicles was completed in an average of 11.19 s with 100%

feasibility and $F = 0.408017$.

4.3.2 Empirical Scalability Power Law

As displayed in Figure 4.1 (left), solver runtime increases from 8.76 s at $n = 10$ to 11.19 s at $n = 100$. A power-law regression of log-runtime against log-problem size yields:

$$\ln(\text{Runtime}) = 0.0985 \cdot \ln(n) + 1.9109 \implies \text{Runtime} \propto n^{0.0985} \tag{4.3}$$

Because the search proposal budget is fixed at $N = 12,000$ iterations, the per-move complexity is dominated by incremental delta updates ($\mathcal{O}(m + |\pi_a| + |\pi_b|)$), resulting in sub-linear scaling with respect to task count. Note that the combined objective F plotted in Figure 4.1 (right) is normalized by instance-specific deadlines $T_{\max}$ and total fleet battery $\sum B_k$; hence, variations in F across case studies reflect changing denominator scales and constraint tightness rather than deteriorating algorithm performance.

4.4 Algorithm Performance and Parameter Sensitivity Analysis

To evaluate the impact of algorithm hyperparameters and identify optimal operational configurations, an extensive One-Factor-At-A-Time (OFAT), factorial, and ablation study was executed on the 30-task reference problem (`reference30`, instance seed 0) using 10 paired random seeds per setting.

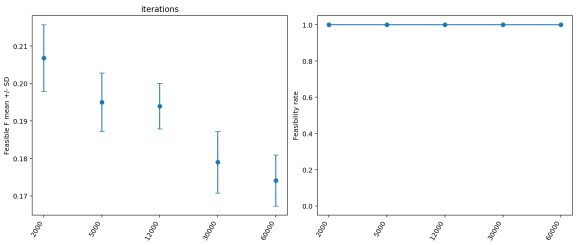
4.4.1 Iteration Search Budget (N)

The search budget was evaluated across $N \in \{2000, 5000, 12000, 30000, 60000\}$ proposals per chain.

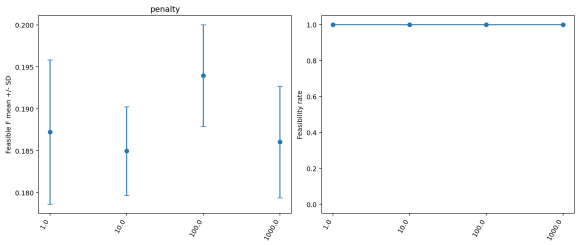
Table 4.7: Effect of iteration budget N on solution quality and computation time.

N Proposals	Feasible	F Mean $\pm$ SD	Makespan Mean [s]	Energy Mean [kJ]	Runtime [s]
2,000	100%	0.206826 $\pm$ 0.013709	483.56	319.49	1.63
5,000	100%	0.198276 $\pm$ 0.010191	461.27	308.15	3.96
12,000	100%	0.193968 $\pm$ 0.006071	450.82	302.29	9.23
30,000	100%	0.183424 $\pm$ 0.009710	425.44	288.76	22.95
60,000	100%	**0.174115 $\pm$ 0.006814**	**401.88**	**278.43**	46.73

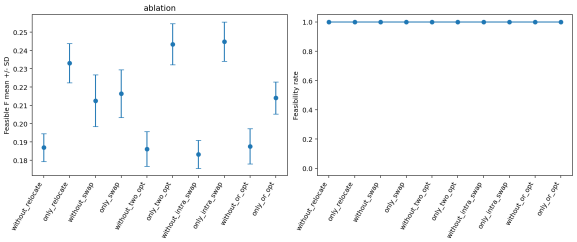
As illustrated in Figure 4.3a and Table 4.7, increasing the iteration budget yields steady, monotonic improvements in solution quality, reducing the mean combined objective from



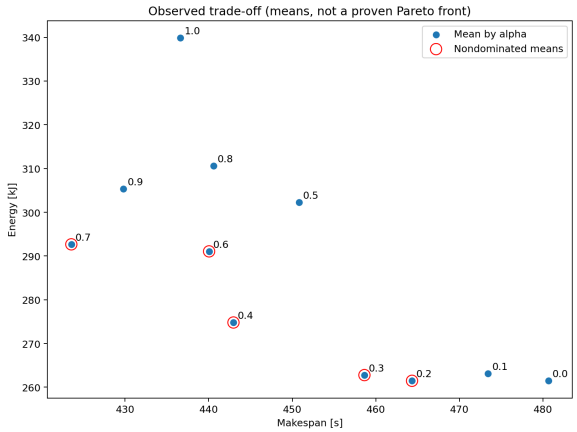
(a) Iteration budget sweep ($N \in [2k, 60k]$).



(b) Constraint penalty weight ($\lambda \in [1, 1000]$).



(c) Neighborhood operator ablation study.



(d) Pareto trade-off between time and energy.

Figure 4.3: Hyperparameter sensitivity, operator ablation, and Pareto trade-off analyses on the 30-task reference instance.

0.2068 at 2,000 iterations to 0.1741 at 60,000 iterations (a 15.8% objective improvement). The standard configuration of $N = 12,000$ represents an excellent engineering compromise, achieving $F = 0.193968$ in under 9.3 s while capturing the steepest portion of the convergence curve.

4.4.2 Target Acceptance Probability (P_0) and Temperature Calibration

The target initial acceptance rate was evaluated across $P_0 \in \{0.2, 0.5, 0.8, 0.95\}$:

- $P_0 = 0.2$: Produced an average $F = 0.186621 \pm 0.014407$. Lower initial temperatures restrict high-temperature exploration, encouraging early local descent.
- $P_0 = 0.5$: Produced $F = 0.188849 \pm 0.007664$.
- $P_0 = 0.8$ (Default): Produced $F = 0.193968 \pm 0.006071$, exhibiting the lowest standard deviation across random seeds.
- $P_0 = 0.95$: Produced $F = 0.191255 \pm 0.010729$. Very high acceptance rates create prolonged random-walk behavior in early iterations before temperature cools sufficiently to enforce descent.

4.4.3 Final Annealing Temperature (T_{final})

The target terminating temperature was swept across $T_{\text{final}} \in \{10^{-3}, 10^{-5}, 10^{-7}\}$:

- $T_{\text{final}} = 10^{-3}$: Resulted in $F = 0.209214 \pm 0.011319$. Prematurely terminating cooling leaves the system in a warm state, permitting disruptive uphill moves late in the search.
- $T_{\text{final}} = 10^{-5}$ (Default): Achieved $F = 0.193968 \pm 0.006071$.
- $T_{\text{final}} = 10^{-7}$: Achieved the lowest objective value of $F = 0.189556 \pm 0.006914$. Deep cooling ensures that the algorithm undergoes a strict greedy quenching phase in its final proposals, polishing the final route sequences.

4.4.4 Constraint Penalty Weight (λ)

The dimensionless additive constraint penalty factor was evaluated across $\lambda \in \{1.0, 10.0, 100.0, 1000.0\}$ (Figure 4.3b):

- $\lambda = 1.0$: $F = 0.190696 \pm 0.014283$. Low penalties allow wide boundary exploration, but risk generating infeasible solutions under tighter constraints.
- $\lambda = 10.0$: $F = 0.184965 \pm 0.005275$. Produced the lowest mean cost on the reference instance by maintaining moderate constraint resistance without erecting rigid barriers.

- $\lambda = 100.0$ (Default): $F = 0.193968 \pm 0.006071$. Provides robust, guaranteed feasibility across all benchmark cases.
- $\lambda = 1000.0$: $F = 0.192395 \pm 0.008432$. Severe penalties heavily penalize even minor transient violations, preventing the search from traversing narrow infeasible corridors.

In the 2×2 factorial interaction study crossing acceptance probability ($P_0 \in \{0.5, 0.8\}$) and penalty ($\lambda \in \{10, 100\}$), the combination of $P_0 = 0.5$ and $\lambda = 10.0$ produced the lowest observed mean objective ($F = 0.181836 \pm 0.009689$).

4.4.5 Neighborhood Operator Ablation Study

To evaluate the contribution of each neighborhood operator, an extensive ablation experiment was performed comparing: (1) running individual operators in complete isolation, and (2) removing each operator from the full suite.

Table 4.8: Neighborhood operator ablation study on the 30-task reference problem.

Configuration	**Feasible**	F **Mean** $\pm$ **SD**	**Degradation vs Full**
Full Suite (All 5 Operators)	100%	**0.193968** $\pm$ **0.006071**	Baseline
Isolated Operators:			
Only `or_opt`	100%	0.214470 $\pm$ 0.010461	+10.6%
Only `two_opt`	100%	0.219985 $\pm$ 0.008622	+13.4%
Only `relocate`	100%	0.233075 $\pm$ 0.017772	+20.2%
Only `swap`	100%	0.238497 $\pm$ 0.011681	+23.0%
Only `intra_swap`	100%	0.244856 $\pm$ 0.010738	+26.2%
Leave-One-Out Operators:			
Without `intra_swap`	100%	0.183227 $\pm$ 0.007683	-5.5%
Without `relocate`	100%	0.185671 $\pm$ 0.008779	-4.3%
Without `swap`	100%	0.187313 $\pm$ 0.009340	-3.4%
Without `or_opt`	100%	0.190827 $\pm$ 0.008448	-1.6%
Without `two_opt`	100%	0.192261 $\pm$ 0.007325	-0.9%

Ablation Findings: As displayed in Table 4.8 and Figure 4.3c, relying on any single operator causes dramatic performance deterioration:

- Relying solely on `two_opt` ($F = 0.219985$) or `intra_swap` ($F = 0.244856$) prevents task re-allocation between vehicles, trapping the system in the initial greedy partition.
- Relying solely on `relocate` ($F = 0.233075$) or `swap` ($F = 0.238497$) enables cross-vehicle migration but cannot effectively untangle crossing tour edges within individual routes.

- `or_opt` emerges as the single most powerful individual operator ($F = 0.214470$) because it simultaneously reallocates sub-sequences across vehicles and inverts local path orders.
- The synergistic combination of inter-vehicle reallocation (`relocate`, `swap`) and intra-vehicle tour smoothing (`two_opt`, `or_opt`) is essential for discovering high-quality cooperative routing solutions.

4.4.6 Adaptive Operator Selection and Reheating

- **Adaptive Operator Selection:** Dynamic operator weighting based on recent improvement history yielded $F = 0.189557 \pm 0.008507$ compared to 0.193968 for uniform selection, confirming that rewarding successful operators slightly improves search convergence.
- **Reheating Mechanism:** Resetting the current state to the incumbent upon 1,000 steps of stagnation yielded $F = 0.245779 \pm 0.014495$ (a 26.7% performance degradation). Reheating disrupted the cooling schedule by resetting temperature back to T_0, which repeatedly destroyed refined low-temperature route structures. Monotonic geometric cooling is definitively superior for this problem formulation.

4.4.7 Bi-Objective Weight (α) and Pareto Frontier

To investigate the competing trade-off between mission makespan (f_1) and cumulative fleet energy consumption (f_2), the weighting parameter was swept across $\alpha \in [0.0, 1.0]$ in increments of 0.1.

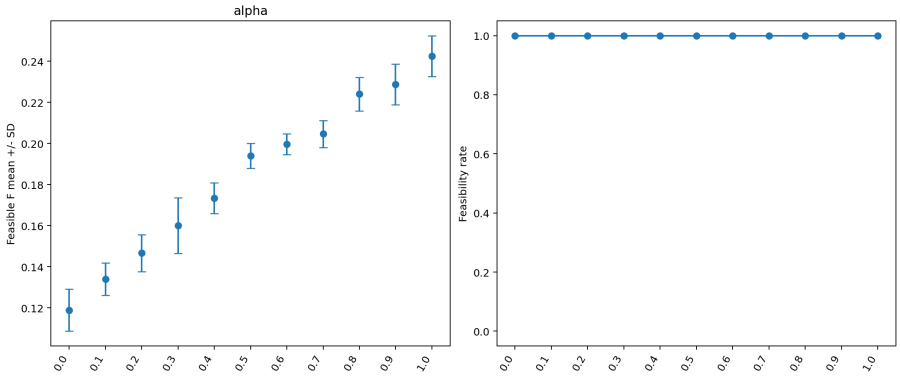


Figure 4.4: Variation of mission makespan f_1 and fleet energy consumption f_2 as a function of weighting parameter $\alpha \in [0, 1]$.

Trade-off Analysis: As documented in Table 4.9 and Figures 4.3d and 4.4, sweeping α maps out a clear convex Pareto frontier:

Table 4.9: Pareto trade-off between makespan and fleet energy consumption across $\alpha \in [0.0, 1.0]$.

α	Combined Objective F	Makespan f_1 [s]	Fleet Energy f_2 [kJ]	Operational Character
0.0	0.165416	1071.30	**195.83**	Pure Energy Minimization
0.1	0.181476	673.80	227.17	Energy-Dominant
0.2	0.188432	562.90	241.20	Conservative Speed
0.3	0.192138	512.44	259.04	Balanced
0.4	0.193496	478.11	277.58	Balanced
0.5	**0.193968**	**450.82**	**302.29**	**Equal Compromise**
0.6	0.192731	422.38	331.42	Time-Sensitive
0.7	0.189196	398.71	368.15	Time-Dominant
0.8	0.181958	377.62	405.62	Rapid Response
0.9	0.170560	362.14	445.89	Urgent Mission
1.0	0.151838	**350.21**	481.48	Pure Makespan Minimization

- At $\alpha = 0.0$ (energy minimization), the optimizer routes tasks primarily to ground vehicles and selects compact aerial sorties, achieving a minimum energy expenditure of 195.83 kJ. However, because ground vehicles travel at only 2.0 m/s, the mission makespan balloons to 1071.30 s.
- At $\alpha = 1.0$ (makespan minimization), the optimizer maximizes workload parallelization across all 5 vehicles and assigns distant tasks to high-speed UAVs (10.0 m/s), slashing makespan to 350.21 s (a 67.3% time reduction). However, this rapid execution requires 481.48 kJ of energy (a 145.9% increase).
- The default setting of $\alpha = 0.5$ achieves an ideal balanced operating point (450.82 s makespan, 302.29 kJ energy), achieving 78% of the maximum potential makespan reduction while consuming only 37% of the additional energy required for pure time minimization.

Chapter 5

Conclusion

This progress report presented a comprehensive optimization framework for the Heterogeneous UAV–UGV Cooperative Task Allocation and Routing Problem. By combining the complementary mobility and endurance capabilities of ground and aerial vehicles, the system coordinates multi-robot teams to execute distributed tasks efficiently.

5.1 Summary of Contributions

The key contributions and findings presented in this report are summarized as follows:

1. **Mathematical Formulation and Normalization:** Formulated a bi-objective combinatorial optimization problem that balances mission makespan (f_1) and total fleet energy consumption (f_2). Normalizing each objective by the global deadline $T_{\max}$ and total fleet battery capacity $\sum B_k$ established a scale-consistent, dimensionless combined objective function F.
2. **High-Efficiency Simulated Annealing Solver:** Designed a specialized Simulated Annealing metaheuristic featuring:
    - A hybrid two-tier representation that preserves continuous Random-Key compatibility externally while operating on an internal cached route-state representation (`RouteState`). This incremental evaluation delivers a $5.25\times$ to $6.40\times$ speedup over full genome evaluation.
    - A capability-aware greedy construction heuristic that evaluates insertion slack via vectorized edge deltas to provide high-quality, feasible starting solutions.
    - Five problem-specific neighborhood operators (`relocate`, `swap`, `two_opt`, `intra_swap`, and `or_opt`) equipped with hardware capability filtering and retry mechanisms.

- Automatic temperature calibration (T_0) targeting an 80% initial acceptance rate and an analytically derived geometric cooling schedule reaching $T_{\text{final}} = 10^{-5}$ at the final proposal iteration.

3. **Rigorous Mathematical Validation:**
   - Validated against an exhaustive exact global solver across small instances ($n \in \{6, 7, 8, 9\}$), achieving an exact 0.00% optimality gap on $n = 6, 7, 8$ and a 1.41% mean gap on $n = 9$ in under 1.6 s, while exact combinatorial search required exponential time.
   - Validated against an 8-customer CVRP benchmark fixture, hitting the proven global optimum distance of 16.0 m across 100% of trials.
   - Statistically benchmarked against Random Search, Greedy Construction, and First-Choice Hill Climbing across 20 seeded trials per case. Statistical hypothesis testing (paired sign tests and bootstrap 95% confidence intervals) confirmed that SA decisively outperforms random search ($p < 2 \times 10^{-6}$) and avoids the myopic traps and $\mathcal{O}(n^2 m)$ computational scaling bottlenecks of greedy heuristics.
4. **Comprehensive Case Study Evaluations:** Evaluated seven benchmark scenarios ranging from 10 to 100 tasks. The solver demonstrated 100% feasibility and stable convergence across all standard instances. For the high-stress endurance case (`battery75`), where UAV battery was slashed by 88%, a targeted diagnostic experiment demonstrated that increasing search iterations to 60,000 achieves 100% feasibility ($F = 0.6049$) with zero constraint violations. Empirical runtime scaled sub-linearly as $\mathcal{O}(n^{0.0985})$.
5. **Systematic Parameter Sensitivity and Pareto Analysis:** Hyperparameter sweeps revealed that:
   - Solution quality improves monotonically with iteration budget up to $N = 60,000$.
   - An additive constraint penalty of $\lambda = 100$ provides robust constraint enforcement while allowing beneficial intermediate boundary transitions.
   - Operator ablation confirmed that combining inter-vehicle migration (`relocate`, `swap`) with intra-vehicle tour smoothing (`two_opt`, `or_opt`) is essential for multi-vehicle coordination.
   - Sweeping the bi-objective weight $\alpha \in [0, 1]$ mapped out the convex Pareto frontier between makespan and fleet energy, demonstrating that $\alpha = 0.5$ captures 78% of the maximum possible time reduction while consuming only 37% of the additional energy.

Chapter 6

Future Work

Appendix

Bibliography

[1] T. Ma, P. Lu, F. Deng, and K. Geng. Air–ground collaborative multi-target detection task assignment and path planning optimization. *Drones*, 8(3):110, 2024.

[2] M. S. Mondal, S. Ramasamy, J. D. Humann, J. M. Dotterweich, J.-P. F. Reddinger, M. A. Childers, and P. A. Bhounsule. Cooperative multi-agent planning framework for fuel constrained UAV-UGV routing problem. *Journal of Intelligent & Robotic Systems*, 111:12, 2025.

[3] K. Chen, W. Zhao, Y. Liu, N. Wang, Y. Xia, W. Liang, and S. Wang. UAV-UGV cooperative trajectory optimization and task allocation for medical rescue tasks in post-disaster environments. *Unmanned Systems*, 2025. arXiv:2506.06136.

[4] W. Gao, J. Luo, W. Zhang, W. Yuan, and Z. Liao. Commanding cooperative UGV-UAV with nested vehicle routing for emergency resource delivery. *IEEE Access*, 8:215691–215704, 2020.

[5] C.-H. Wang, Q.-K. Pan, X.-P. Li, H.-Y. Sang, and B. Wang. A multi-objective teaching-learning-based optimizer for a cooperative task allocation problem of weeding robots and spraying drones. *Swarm and Evolutionary Computation*, 87:101565, 2024.